

Final Project: House Price Prediction

Lucy, Carter, Connor, Elliot, George

Problem Description

This project focuses on predicting housing prices in the United States using a dataset that includes information about individual properties. Each home is described by features such as size, number of bedrooms and bathrooms, condition, and location. The goal is to use these characteristics to estimate the price of a house as accurately as possible.

This is a supervised learning problem, specifically a regression task, because the target variable is a continuous numerical value. The model is trained on historical data where both the features and the corresponding sale prices are known. By learning patterns in this data, the model can then make predictions for new properties.

Predicting housing prices is a meaningful real-world problem. Accurate estimates can help buyers understand fair market value, assist sellers in setting competitive prices, and support real estate professionals and financial institutions in making informed decisions. However, the problem is challenging because housing prices are influenced by many factors, some of which are not fully captured in the dataset. For example, aspects like neighborhood quality, local economic conditions, and market trends may play a significant role but are difficult to quantify.

Another challenge is that housing data often contains variability and outliers. High-end properties can have prices far above the average, which can make it harder for models to generalize. Additionally, location-based features such as city or zip code can be complex to represent effectively in a model. Because of these factors, building a model that balances accuracy and generalization requires careful preprocessing and thoughtful feature selection.

Overall, the objective of this project is not only to build a model that predicts housing prices, but also to understand which factors most strongly influence those prices and evaluate how well the model performs on unseen data.

Dataset

The USA House Prices dataset, sourced from Kaggle (fratzcan/usa-house-prices), contains 4,140 house sale records across 18 original features. The numeric features include the target variable price (sale price in USD), along with sqft_living, sqft_lot, sqft_above, bedrooms, bathrooms, yr_built, and yr_renovated, while the categorical and ordinal features cover date, waterfront, view, condition, floors, street, city, and statezip.

Preprocessing began with a data quality check. No missing values were found, though imputation was still built into the pipeline as a safeguard, and duplicates were screened for. The date column was parsed into two numeric features, sale_year and sale_month, after which the original date string was dropped. The street column was removed due to high cardinality (overfitting risk), and the country was dropped as a constant value. Price outliers were then trimmed by removing the bottom 2% and top 1% of prices, reducing the dataset from 4,140 to approximately 4,016 rows.

A standard 80/20 train/test split was applied with random_state=42, and a ColumnTransformer pipeline was fit exclusively on the training set (applying SimpleImputer + StandardScaler to numeric columns and SimpleImputer + OneHotEncoder to the categorical columns city and statezip), producing a final processed feature space of 132 dimensions by 4016 records.

Models

Two separate models were evaluated for this task, however we started with linear regression as a basic, easy to implement baseline.

Once the data was fully preprocessed, we first scaled the data, so that weights with much higher scales would not dominate the loss function during training. Next, we imported scikit learn, and created an sklearn LinearRegression instance. We then trained on the scaled training data and our test target. Behind the scenes, sklearn uses optimized numerical methods like SVD or some form of gradient descent. Next, we predicted using the scaled test data, and stored the predictions. We then used a custom evaluate_regression formula to output MSE, RMSE, MAE, and R^2 . After that, we stored the linear regression, sorted them using absolute value, and returned both the most important feature names and their respective coefficient values. It appeared that the largest coefficients in absolute value belonged to specific zip codes and cities, which makes sense intuitively.

For this model, we can interpret how each feature affects the prediction. In the linear regression model, there is also a lack of complexity in that no custom functions must be written for setup or training.

Upon deliberation, we also chose to create and evaluate a neural network. This represented a more complex task than linear regression. For our neural network, we used pytorch, tensor datasets, and dataloaders. As in linear regression, our first step was to scale the data. This would again help from large scales dominating the loss function during training, resulting in more optimal training. Next, we worked to define the custom functions. We defined the NN class itself with a forward pass function, then wrote the train and test functions. Additionally, we created data loaders. The dimensions of our NN's layers were 132 (input), 128 (hidden), 64 (hidden), and 1 for a continuous regression output. We next specified our loss function to be MSE and our optimizer to be the Adam optimizer. We then trained over 10 epochs. After training, we got predictions in the scaled test data, reversing at the end to get predictions in the correct units. Then, we used our evaluate function to again print the needed regression metrics (MSE, RMSE, MAE, and R^2).

Results and Design Choices

The final results showed that both the Linear Regression model and the Neural Network model were able to predict housing prices reasonably well after preprocessing and outlier removal. Originally, both models produced much lower R^2 values, with the Linear Regression model around 0.5 and the Neural Network around 0.6. After observing the predicted vs. actual price plot for the neural network model, we noticed that there were several extreme housing prices that we thought might be negatively affecting model performance. Because of this, one of the most important design choices in the project was removing those price outliers. The final approach removed the bottom 2% and top 1% of housing prices from the dataset. This significantly improved the overall fit of both models and increased the R^2 values to around 0.8. In making this change, we had to acknowledge that our model is ideal for serving the vast majority of homebuyers, however it would not be advisable for someone looking to buy a house in those extreme high or low price ranges.

Even though the Linear Regression model slightly outperformed the Neural Network, both models were still included in the final project because they achieved very similar results while using very different approaches. The Linear Regression model achieved an R^2 value of approximately 0.817, while the Neural

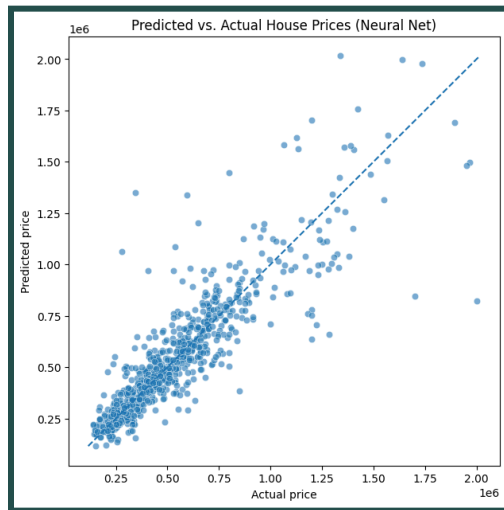
Network achieved an R^2 value of approximately 0.806. The R^2 value measures how much of the variation in housing prices can be explained by the model, so values closer to 1 indicate a stronger overall fit. In the case of house price prediction, an R^2 value of 0.8 is considered fairly good, although we believe that it could be improved with more data points, such as crime rate or school system desirability. The Linear Regression model also achieved the lower Mean Squared Error (MSE) at approximately 16.3 billion, compared to about 17.2 billion for the Neural Network. MSE measures the average squared prediction error, meaning lower values indicate predictions closer to the actual housing prices. Root Mean Squared Error (RMSE) was also used because it converts the error back into dollar units, making the results easier to interpret in a real-world context. The RMSE values were approximately \$127,000 for the Linear Regression model and about \$131,000 for the Neural Network model, meaning predictions were typically off by around that amount on average.

Another important set of design choices involved the structure of the Neural Network itself. The final network used dimensions of $132 \rightarrow 128 \rightarrow 64 \rightarrow 1$. The 132 input features came from both the original quantitative variables and the expanded qualitative variables after one-hot encoding categorical columns such as city and statezip. This allowed qualitative location information to be “flushed out” into many separate numerical input columns that the model could process more effectively. From there, the hidden layers gradually reduced the dimensionality while learning more complex relationships between features and housing prices.

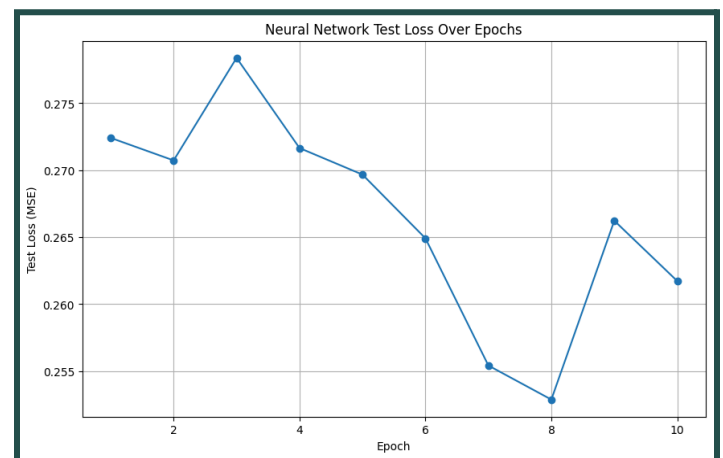
Additional design choices were also made to produce the best overall model fit. A batch size of 16 produced the strongest accuracy during testing, indicating that we should choose that value. The model was trained for 10 epochs because this balanced underfitting and overfitting. Too few epochs prevented the model from learning enough patterns, while too many risked memorizing the training data instead of generalizing well to unseen houses.

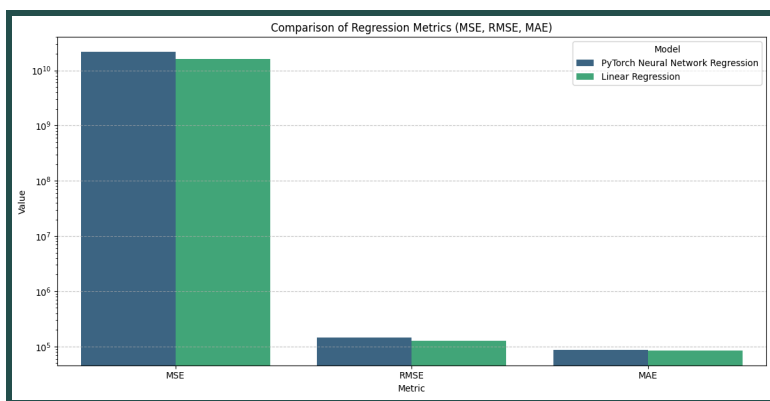
Graphs and Visualizations

These scatter plots compare our model's predicted house prices against the actual selling prices. The diagonal line represents perfect predictions where predicted equals actual. For the most part, the data is concentrated around this line, with the exception of some outliers.

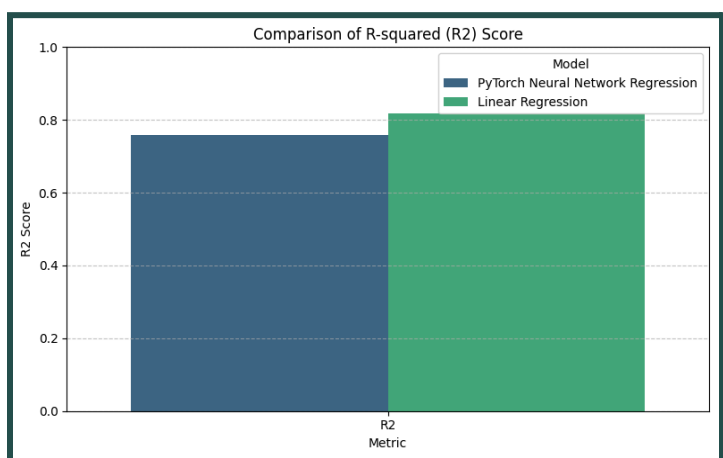


The line plot to the right tracks the Mean Squared Error (MSE) on the test dataset as our neural network trains over several epochs (10). A steady decrease in loss indicates that the model is learning and improving. Aside from a jump at the 8-9 epoch mark, which could potentially be attributed to getting stuck in a local minima, our model's loss becomes lower over the course of 10 iterations.

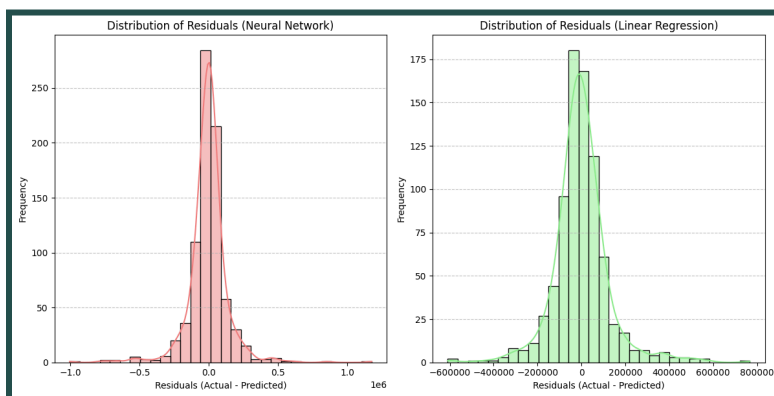




These metrics quantify the accuracy of our model. Lower values for MSE, RMSE, and MAE indicate better model performance, as they represent smaller average differences between predicted and actual values.



R-squared tells us how much of the variation in house prices our model can explain. A higher R^2 (closer to 1) is better, as it means our model is doing a great job of explaining the price changes.



These histograms display the distribution of our models' prediction errors (residuals). For a good model, residuals should ideally be normally distributed and centered around zero.

Conclusion

Overall, this project demonstrated how AI models can be used to make meaningful predictions in a real-world setting. While both the Linear Regression and Neural Network models performed well, the results also showed the importance of preprocessing, feature selection, and handling outliers in improving model accuracy. In the future, adding more detailed data related to neighborhoods, schools, or local economic conditions could further improve performance and help create even more reliable housing price predictions.

Links

Dataset: [USA House Prices](#)

Link to Colab: [!\[\]\(9dfdaff1d86ba3c1f8353b4d1b61b8c5_img.jpg\) Updated ai_main_project.ipynb](#)